Praktikum 2

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Prof. Ir. Dadet Pramadihanto, M.Eng, Ph.D
MATA KULIAH	: Praktikum Pengolahan Sinyal dan Citra
TANGGAL	: Kamis, 27 Februari 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 2
Recording Voice Signal from Microphone and its Playback Codes

1. DASAR TEORI

Pemrosesan sinyal digital pada sinyal suara adalah teknik mengubah gelombang suara analog menjadi bentuk digital agar dapat dianalisis, dimanipulasi, dan digunakan dalam berbagai aplikasi. Proses ini dimulai dengan sampling, yaitu mengambil sampel amplitudo suara pada interval waktu tertentu sesuai dengan sampling rate (misalnya 44.1 kHz pada kualitas CD audio), serta kuantisasi, yang mengubah nilai amplitudo kontinu menjadi data diskrit berdasarkan bit-depth tertentu. Semakin tinggi nilai sampling rate dan bit-depth, semakin baik kualitas suara yang dapat direproduksi. Setelah sinyal dikonversi ke bentuk digital, data suara disimpan dalam format seperti WAV, yang memungkinkan penyimpanan tanpa kompresi sehingga setiap detail suara tetap utuh. Data yang tersimpan ini nantinya dapat digunakan untuk berbagai proses lebih lanjut, seperti analisis suara, penghapusan noise, atau pengolahan efek audio.

Dalam pemrosesan sinyal, manipulasi dan analisis sinyal menjadi tahap penting untuk memahami karakteristik suara. Filtering digunakan untuk menghilangkan frekuensi yang tidak diinginkan, sementara normalisasi dapat menyesuaikan level volume agar lebih konsisten. Salah satu teknik analisis yang sering digunakan adalah FFT, yang mengubah sinyal dari domain waktu ke domain frekuensi untuk melihat distribusi spektrumnya. Ini berguna untuk mengenali pola suara, ekstraksi fitur, atau bahkan dalam aplikasi kecerdasan buatan seperti pengenalan suara. Selain itu, sinyal dapat divisualisasikan dalam bentuk waveform untuk menunjukkan bagaimana amplitudo berubah terhadap waktu, sehingga pola tertentu bisa diidentifikasi dengan lebih mudah. Visualisasi ini sering digunakan dalam debugging atau pemrosesan lebih lanjut pada sinyal suara untuk memastikan bahwa data yang diperoleh sesuai dengan kebutuhan.

Selain analisis dan manipulasi sinyal, interaksi dengan sinyal suara dalam aplikasi digital menjadi aspek penting dalam implementasi teknologi berbasis audio. Sistem harus dapat menangkap input dari mikrofon, menyimpan data dengan format yang sesuai, dan memberikan keluaran berupa rekaman yang dapat diputar kembali atau divisualisasikan. Untuk mendukung interaksi ini, berbagai pustaka pemrograman digunakan untuk mengontrol proses rekaman dan pemutaran secara real-time, memungkinkan pengguna untuk menangani data suara secara efisien.

2. PROGRAM

1. Program untuk merekam suara, menyimpannya sebagai file WAV, kemudian menampilkan dan menyimpan plot gelombang suara dalam bentuk PNG.

sound_record_final.py > ...

```python
import os
import pyaudio
import wave
import numpy as np
import matplotlib.pyplot as plt
import keyboard

# Folder tempat menyimpan rekaman
FOLDER_PATH = "D:\\1. KULIAH\\Semester IV\\4. Sinyal dan Citra\\record"
os.makedirs(FOLDER_PATH, exist_ok=True)

# Kata-kata yang akan direkam
words = ["bawah", "bukan", "jika", "lama", "memiliki", "rendah", "saat", "samping"] # Bisa ditambah hingga 100

# Konfigurasi rekaman
chunk = 1024
sample_format = pyaudio.paInt16
channels = 2
fs = 44100  # 44.1 kHz
p = pyaudio.PyAudio()

def record_audio(filename):
    print(f"Merekam \"{filename}\". Tekan ENTER untuk berhenti...")

    stream = p.open(format=sample_format,
                    channels=channels,
                    rate=fs,
                    frames_per_buffer=chunk,
                    input=True)

    frames = []

    while True:
        data = stream.read(chunk, exception_on_overflow=False)
        frames.append(data)
        if keyboard.is_pressed("enter"):
            keyboard.wait("enter")  # Pastikan ENTER benar-benar ditekan sebelum berhenti
            break

    stream.stop_stream()
    stream.close()

    filename_path = os.path.join(FOLDER_PATH, f"{filename}.wav")
    with wave.open(filename_path, 'wb') as wf:
        wf.setnchannels(channels)
        wf.setsampwidth(p.get_sample_size(sample_format))
        wf.setframerate(fs)
        wf.writeframes(b''.join(frames))

    print(f"Rekaman \"{filename}\" disimpan!")
    plot_audio_waveform(filename_path, filename)

# Fungsi mengecek kata yang belum direkam
def check_missing_recordings():
    existing_files = {f.split(".")[0] for f in os.listdir(FOLDER_PATH) if f.endswith(".wav")}
    missing_words = [word for word in words if word not in existing_files]
    return missing_words

# Fungsi untuk membuat plot dan menyimpan sebagai PNG
def plot_audio_waveform(filepath, filename):
    with wave.open(filepath, 'rb') as wf:
        n_channels = wf.getnchannels()
        framerate = wf.getframerate()
        n_frames = wf.getnframes()
        audio_data = wf.readframes(n_frames)

    audio_array = np.frombuffer(audio_data, dtype=np.int16)
    if n_channels == 2:
        audio_array = audio_array[::2]
```

```python

    time_axis = np.linspace(0, len(audio_array) / framerate, num=len(audio_array))

    plt.figure(figsize=(10, 4))
    plt.plot(time_axis, audio_array, color='blue')
    plt.xlabel("Waktu (detik)")
    plt.ylabel("Amplitudo")
    plt.title(f"Gelombang Suara: {filename}")
    plt.grid()

    plot_path = os.path.join(FOLDER_PATH, f"{filename}.png")
    plt.savefig(plot_path)
    plt.close()
    print(f"Grafik gelombang suara {filename} disimpan sebagai PNG!")

# Cek rekaman yang belum ada
missing_words = check_missing_recordings()

if not missing_words:
    print("Semua kata sudah direkam!")
else:
    print("Kata yang belum direkam:", ", ".join(missing_words))
    for word in missing_words:
        record_audio(word)

p.terminate()
print("Program selesai.")
```

2. Program untuk membaca, memutar, dan memplot/visualisasi file WAV.

```python
import wave
import numpy as np
import matplotlib.pyplot as plt
import winsound

filename = input("Masukkan nama file WAV: ")
winsound.PlaySound(filename, winsound.SND_FILENAME)
with wave.open(filename, 'rb') as wf:
    n_channels = wf.getnchannels()
    sample_width = wf.getsampwidth()
    framerate = wf.getframerate()
    n_frames = wf.getnframes()
    audio_data = wf.readframes(n_frames)

audio_array = np.frombuffer(audio_data, dtype=np.int16)
if n_channels == 2:
    audio_array = audio_array[::2]
time = np.linspace(0, len(audio_array) / framerate, num=len(audio_array))

# Plot gelombang suara
plt.figure(figsize=(10, 4))
plt.plot(time, audio_array, label="Gelombang Suara", color="blue")
plt.xlabel("Waktu (detik)")
plt.ylabel("Amplitudo")
plt.title(f"Plot Gelombang Suara dari {filename}")
plt.legend()
plt.grid()
plt.show()
```

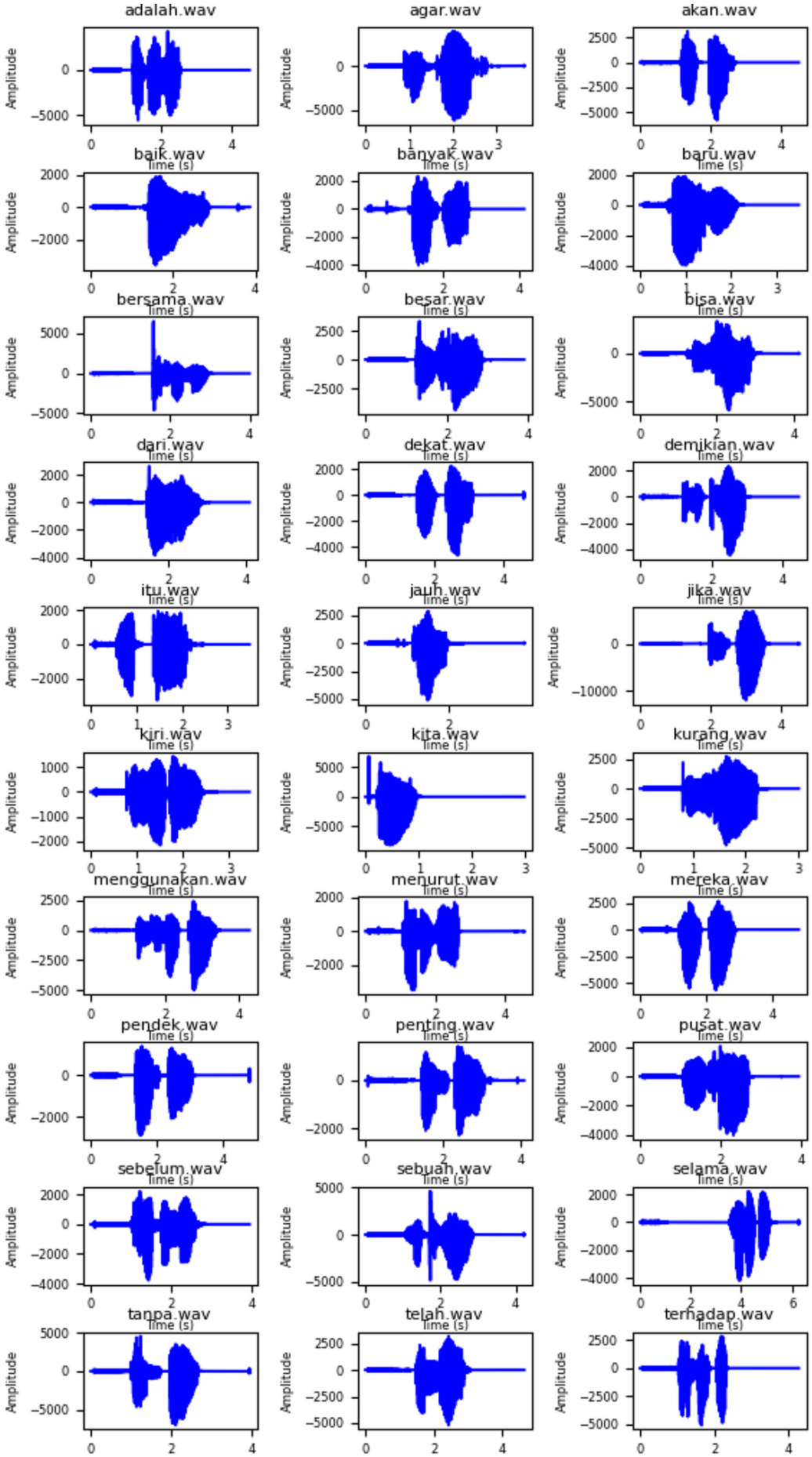
3. Program untuk memplot banyak sinyal audio menjadi 1.

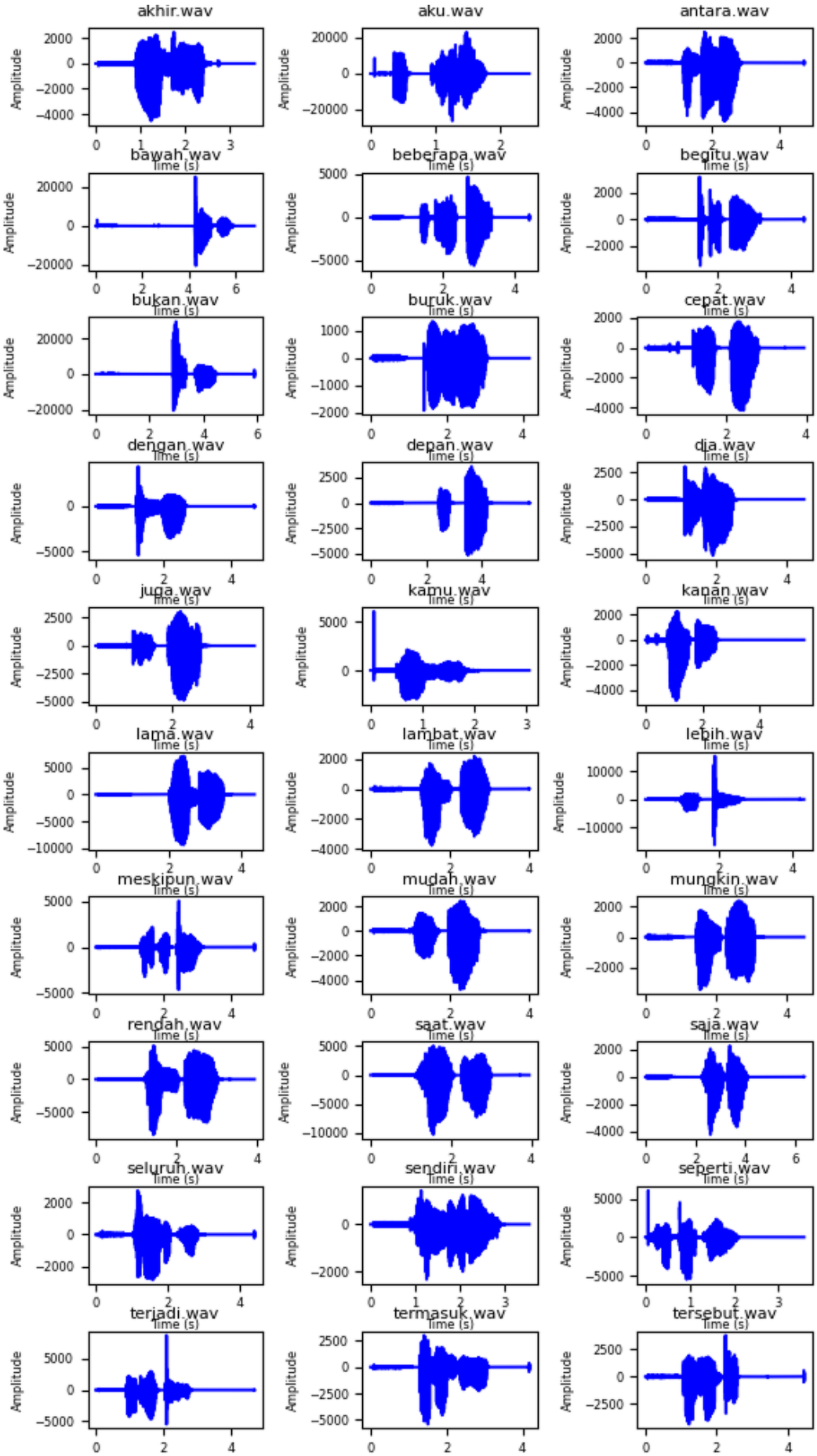
gabung.py > ...

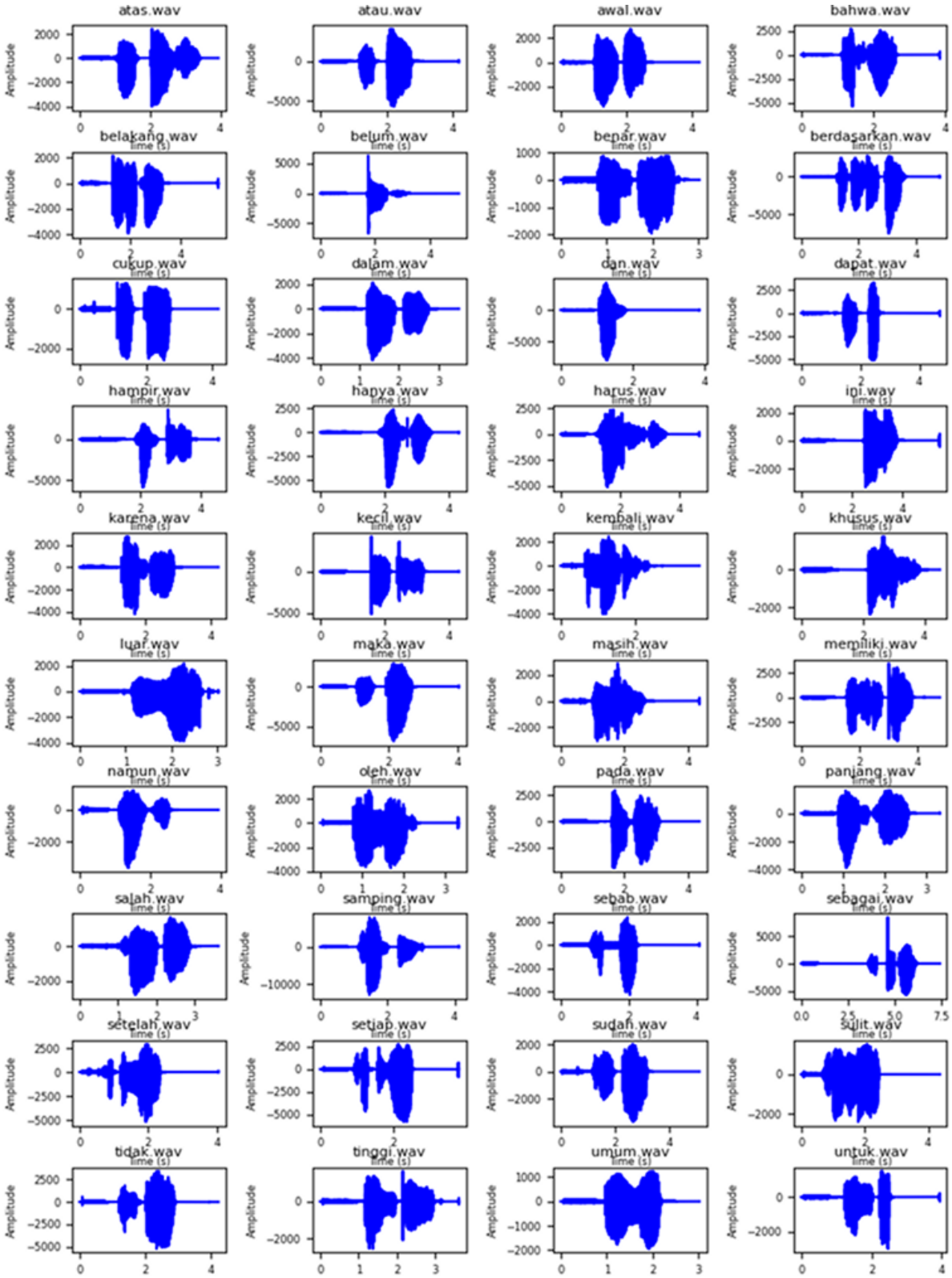
```python
import os
import numpy as np
import matplotlib.pyplot as plt
import wave

# Folder containing the WAV files
FOLDER_PATH = "D:\\1. KULIAH\\Semester IV\\4. Sinyal dan Citra\\record"

def read_wav_file(file_path):
    with wave.open(file_path, 'rb') as wav_file:
        n_frames = wav_file.getnframes()
        frame_rate = wav_file.getframerate()
        audio_data = np.frombuffer(wav_file.readframes(n_frames), dtype=np.int16)
    return audio_data, frame_rate

def plot_wav_files(folder_path):
    wav_files = [f for f in os.listdir(folder_path) if f.endswith(".wav")]
    num_files = len(wav_files)

    if num_files == 0:
        print("No WAV files found in the directory.")
        return

    rows, cols = 10, 10 # Grid layout

    fig, axes = plt.subplots(rows, cols, figsize=(38.4, 21.6))
    axes = axes.flatten()

    for i, wav_file in enumerate(wav_files[:100]):  # Limit to 100 files
        file_path = os.path.join(folder_path, wav_file)
        audio_data, frame_rate = read_wav_file(file_path)
        time_axis = np.linspace(0, len(audio_data) / frame_rate, num=len(audio_data))

        axes[i].plot(time_axis, audio_data, color='blue')
        axes[i].set_title(wav_file, fontsize=8)
        axes[i].set_xlabel("Time (s)", fontsize=6)
        axes[i].set_ylabel("Amplitude", fontsize=6)
        axes[i].tick_params(axis='both', which='both', labelsize=6)

    # Hide unused subplots if there are fewer than 100 files
    for j in range(len(wav_files), rows * cols):
        fig.delaxes(axes[j])

    plt.tight_layout()
    plt.show()
plot_wav_files(FOLDER_PATH)
```

3. HASIL PERCOBAAN

1. File wav dapat diakses pada folder drive berikut: https://drive.google.com/drive/folders/13258o6_bpVuaGnbbh6McfTeXAqV4yNBZ
2. Plot gelombang suara




akhir.wav
aku.wav
antara.wav
bawah.wav
beberapa.wav
begitu.wav
bukan.wav
buruk.wav
cepat.wav
dengan.wav
depan.wav
dia.wav
juga.wav
kamu.wav
kapan.wav
lama.wav
lambat.wav
lebih.wav
meskipun.wav
mudah.wav
mungkin.wav
rendah.wav
saat.wav
saja.wav
seluruh.wav
sendiri.wav
seperti.wav
terjadi.wav
termasuk.wav
tersebut.wav
Amplitude
Time (s)

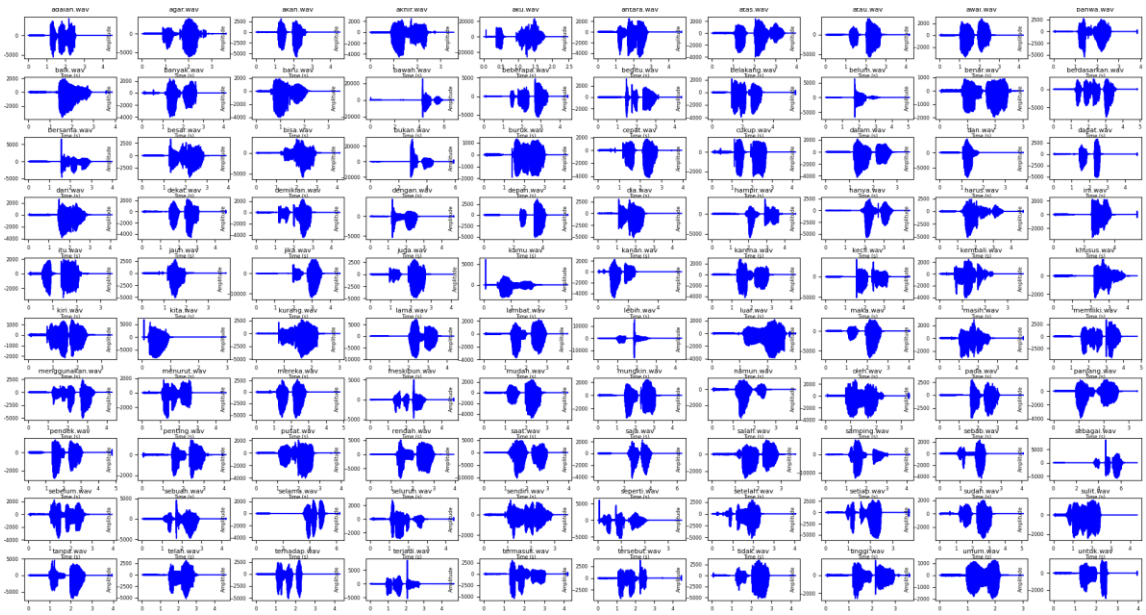

atas.wav
atau.wav
awal.wav
bahwa.wav
belakang.wav
belum.wav
benar.wav
berdasarkan.wav
cukup.wav
dalam.wav
dan.wav
dapat.wav
hampir.wav
hanya.wav
harus.wav
ini.wav
karena.wav
kecil.wav
kembali.wav
khusus.wav
luar.wav
maka.wav
masih.wav
memiliki.wav
namun.wav
oleh.wav
pada.wav
panjang.wav
salah.wav
samping.wav
sebab.wav
sebagai.wav
setelah.wav
setiap.wav
sudah.wav
sulit.wav
tidak.wav
tinggi.wav
umum.wav
untuk.wav
Amplitude
Time (s)

4. ANALISA

Pada praktikum ini, tiga program digunakan secara berurutan untuk merekam, memutar, dan memvisualisasikan sinyal suara dalam bentuk digital. Program pertama digunakan untuk merekam suara dan menyimpannya dalam format WAV, program kedua berfungsi untuk memutar ulang rekaman serta menampilkan bentuk gelombangnya, sementara program ketiga digunakan untuk menampilkan banyak rekaman sekaligus dalam satu tampilan agar lebih mudah dibandingkan. Ketiga program ini saling berkaitan karena membentuk proses lengkap dalam pemrosesan sinyal suara, mulai dari akuisisi data, analisis bentuk gelombang, hingga perbandingan pola suara. Pada program pertama, suara direkam menggunakan pustaka pyaudio dan disimpan dalam format WAV menggunakan pustaka wave. Rekaman dilakukan dengan sampling rate 44.1 kHz, 16-bit depth, dan 2 channel (stereo), yang sesuai dengan standar kualitas audio yang umum digunakan. Program menangkap suara dalam bentuk blok-blok kecil atau bisa disebut sebagai chunks, yang kemudian disusun kembali dan disimpan ke dalam file WAV. Setelah rekaman selesai, program juga memplot gelombang suara menggunakan matplotlib agar dapat terlihat bentuk sinyalnya. Hasilnya menunjukkan bahwa suara berhasil direkam dengan baik, dan bentuk gelombangnya sesuai dengan teori bahwa sinyal suara memiliki pola amplitudo yang bervariasi terhadap waktu.

Program kedua memungkinkan untuk memutar kembali rekaman yang telah dibuat serta menampilkan bentuk gelombangnya secara visual. Dengan menggunakan pustaka wave, program membaca metadata dari file WAV untuk memastikan bahwa data yang digunakan sesuai dengan spesifikasi awal. Data audio kemudian dikonversi ke array numerik menggunakan *numpy.frombuffer()*, yang memungkinkan untuk dimanipulasi dan dianalisis lebih lanjut sehingga jika rekaman memiliki dua kanal (stereo), dapat diambil salah satu kanal untuk mempermudah dalam analisis. Terdapat juga penggunaan *winsound* untuk memutar ulang rekaman sehingga dapat untuk membandingkan hasil audionya dengan bentuk visualnya. Hasilnya menunjukkan bahwa bentuk gelombang yang ditampilkan sesuai dengan suara yang terdengar.

Program ketiga digunakan untuk memvisualisasikan banyak rekaman sekaligus dalam satu tampilan. Ini sangat berguna ketika ingin membandingkan pola gelombang dari berbagai kata yang direkam. Program membaca semua file WAV dalam folder, mengonversi datanya, lalu menampilkannya dalam grid menggunakan *matplotlib*.



Hasilnya menunjukan perbedaan pola suara dari berbagai kata yang direkam dalam satu tampilan, membuatnya lebih mudah untuk dianalisis. Setiap kata memiliki pola gelombang yang unik, sesuai dengan dasar teori di awal bahwa setiap su ara manusia memiliki karakteristik frekuensi yang berbeda.

5. KESIMPULAN

Pemrosesan sinyal suara dalam praktikum ini berhasil menunjukkan bahwa suara bisa direkam, dianalisis, dan divisualisasikan dalam bentuk digital dengan baik. Rekaman yang dihasilkan sesuai dengan teori, di mana bentuk gelombangnya mencerminkan variasi amplitudo suara terhadap waktu. Pemutaran ulang dan tampilan visualnya juga membantu dalam memahami pola sinyal suara yang berbeda. Namun, dalam rekaman masih terdapat gangguan seperti suara kendaraan, percakapan di latar belakang, atau suara lain dari lingkungan sekitar. Gangguan ini bisa memengaruhi analisis, sehingga perlu dilakukan pemrosesan lebih lanjut untuk membersihkan sinyal. Salah satu cara yang bisa digunakan adalah transformasi Fourier, seperti DFT atau DTFT, yang dapat memisahkan komponen frekuensi dalam sinyal. Dengan teknik ini, suara mentah sebelumnya dapat disaring noisenya dan dipertahankan bagian suara yang diinginkan, sehingga hasil analisis menjadi lebih akurat dan jelas.